MD-Kivy: An Interactive Educational Visualization for Exploring Molecular Dynamics

Anastasiia Brund* Jared Galindo Hernandez† Iris Song‡ Ayon Saneel Das§ Craig Jansen¶ Anne Bowen‖

The University of Texas at Austin Texas Advanced Computing Center

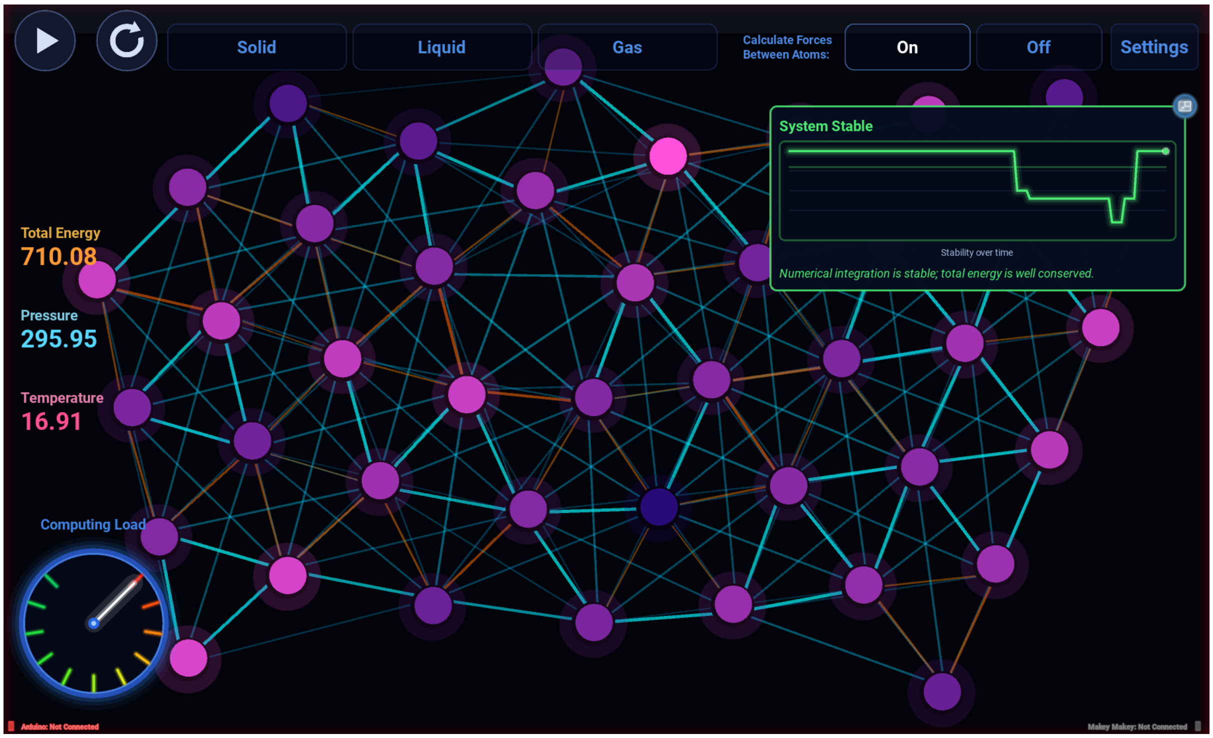

Figure 1: MD-Kivy single-player exploration mode showing real-time molecular motion, force controls, thermodynamic measurements, and computational load feedback.

Abstract

Molecular dynamics (MD) simulations play a central role in modern chemistry, biology, and materials science, yet the computational processes underlying these simulations remain largely invisible to students and the public. Existing educational molecular dynamics tools are primarily designed for classroom or individual desktop use and rarely support collaborative public engagement or illustrate the computational principles behind simulation. We present MD-Kivy, an open-source educational visualization designed for science centers and outreach environments that enables visitors to explore both molecular behavior and the computational principles underlying molecular dynamics through direct interaction. MD-Kivy employs a Lennard-Jones particle simulation that emphasizes conceptual understanding rather than atomistic accuracy. Users manipulate simulation parameters to investigate how intermolecular forces influence particle motion and emergent collective behavior in real time. Color-mapped molecular velocities and a live computational workload display help users connect microscopic dynamics, emergent macroscopic behavior, and the computational scaling of pairwise force calculations. To encourage collaborative exploration, the system also includes a multiplayer mode in which groups of participants compete on opposite sides of a shared screen while optional tangible controllers map physical actions to molecular motion. MD-Kivy demonstrates a design approach for public educational visualization that integrates molecular science, computational thinking, and collaborative learning within a single exhibit experience.

Index Terms: Molecular dynamics, STEM education, interactive simulation, touchscreen interfaces, physics visualization.

*e-mail: asb4247@my.utexas.edu
†e-mail:jg73794@my.utexas.edu
‡e-mail:ritz.m.song@gmail.com
§e-mail:ayon.s.das@hotmail.com
¶e-mail:cjansen@tacc.utexas.edu
‖e-mail:adb@tacc.utexas.edu

1 Introduction

Modern scientific discoveries increasingly depend on computational simulations that are invisible to most learners. Although students encounter the results of molecular dynamics simulations in contexts ranging from drug discovery to materials science, the underlying computational models remain abstract and difficult to understand. Interactive visualization provides an opportunity to bridge this gap by allowing users to manipulate simulation parameters and immediately observe the consequences of their choices.

Molecular dynamics (MD) simulations have become indispensable tools in chemistry, biology, materials science, and drug discovery by modeling the interactions of atoms and molecules over time. Because these simulations compute enormous numbers of pairwise interactions across millions of timesteps, they often require high-performance computing resources. Yet despite their importance, most students and members of the public have little understanding of how these simulations operate or why they require supercomputers. Making these computational demands visible relates to computational thinking, the view that reasoning in computational terms is a valuable skill for everyone, not only computer scientists [23].

Interactive visualization offers a way to transform users from passive observers into active participants. Rather than watching precomputed animations, users manipulate simulation parameters—including particle count and interaction strength—and immediately observe how microscopic interactions produce emergent macroscopic behavior while revealing the computational scaling of the simulation itself. By directly exploring the relationship between molecular behavior and computational workload, molecular simulations become exploratory learning environments rather than passive demonstrations. This approach is especially valuable in informal learning spaces such as visitor centers and museums, where exhibits must support short, self-directed, shared interactions for visitors with diverse ages, backgrounds, and levels of scientific knowledge. Studies of informal science education describe how museums and similar venues promote learning through open-ended, hands-on exploration outside the traditional classroom [14].

Designing educational visualizations for informal learning environments requires different priorities than designing research software. Existing molecular dynamics and molecular visualization tools are typically designed for researchers or classroom instruction and often assume individual desktop use. In contrast, public outreach settings require systems that are legible at a distance, robust under repeated use, easy to operate with little or no instruction, and capable of engaging multiple visitors simultaneously. MD-Kivy was designed around these requirements, combining a qualitative two-dimensional molecular dynamics simulation with a large-format touchscreen interface, real-time parameter controls, visual encodings of molecular behavior and computational workload, and optional tangible controllers for embodied interaction. Rather than striving for atomistic fidelity, MD-Kivy intentionally favors conceptual understanding, enabling visitors to explore the relationships between intermolecular forces, emergent behavior, and computational scaling through direct interaction.

This paper presents MD-Kivy[1], an open-source educational visualization for informal STEM learning. Our contributions are threefold: (1) a collaborative molecular dynamics visualization designed specifically for public-facing touchscreen exhibits; (2) visual representations that connect molecular behavior with computational scaling; and (3) an open-source platform supporting tangible interaction and deployment in museums, visitor centers, and outreach environments. A student with a low attention span can simply play, while an educator can learn advanced concepts all on the same interface.

2 Related Work

Interactive molecular dynamics visualizations enable users to investigate how macroscopic thermodynamic behavior emerges from microscopic particle interactions through direct manipulation of simplified molecular systems [18, 5, 1]. Schroeder demonstrated that simplified two-dimensional Lennard-Jones simulations allow users to investigate phase transitions, diffusion, Brownian motion, thermal equilibrium, and the relationship between microscopic dynamics and macroscopic observables through direct experimentation [18]. Similar educational environments include the Open Source Physics *LJFluidApp* [5], Boston University's *Virtual Molecular Dynamics Laboratory* [4], and the widely used *PhET States of Matter* simulation [1]. Collectively, these systems demonstrate the effectiveness of interactive visualization for supporting conceptual learning through experimentation. However, they are primarily intended for classroom instruction or individual computer use rather than collaborative, public-facing exhibits.

A complementary body of work has focused on immersive molecular visualization and interactive molecular manipulation as research tools. Research systems such as *Narupa* (now Nanover) [16], *InteraChem* [19], *UnityMol* [13], *ChimeraX* with virtual reality support [8], and Interactive Molecular Dynamics (IMD) [21] allow users to inspect molecular structures, manipulate molecules directly, and in some cases steer molecular dynamics simulations in real time. These systems demonstrate the potential of immersive visualization for understanding molecular structure and dynamics, but they primarily support scientific research and advanced instruction. These applications target university students, researchers, or domain experts and generally emphasize atomistically accurate molecular structures rather than simplified conceptual models. Game-based and simulation-based learning also has a long record in chemistry education. Molecular-level simulations have been used as classroom teaching tools for decades [17], and educational games in chemistry date back a full century [6]. Reviews of this work find that game-based learning, gamification, and serious games tend to improve student motivation, engagement, and achievement [7], and the serious-games literature provides design guidance for using them in education [11]. The competitive mode in MD-Kivy follows this approach, using competitive play to make free expansion and gas behavior engaging for public audiences.

While museums and science centers frequently employ interactive exhibits to communicate scientific concepts, comparatively little work has explored how molecular dynamics can be adapted to collaborative, public-facing exhibits that simultaneously illustrate both molecular behavior and computational processes. MD-Kivy builds on these previous efforts while addressing a distinct design space. Rather than prioritizing atomistic fidelity or supporting scientific analysis, the system was designed specifically for informal STEM education in a public visitor center. It combines a qualitative molecular dynamics simulation with real-time parameter exploration, visualizations of molecular behavior and computational workload, multiplayer interaction, and optional tangible interfaces. Together, these features communicate both the physical principles governing molecular motion and the computational principles that make large-scale molecular dynamics simulations possible.

3 System Overview

MD-Kivy is an interactive molecular dynamics visualization application designed for large-format touchscreen displays. MD-Kivy is designed for classroom and outreach use, where users can add particles, adjust parameters, and observe the simulation directly on a shared touchscreen. The application is implemented in Python using the Kivy framework and renders the simulation through the Kivy OpenGL canvas instructions.

The application is organized around two main user-facing modes. The first is a single-player exploration mode, shown in Figure 1, where users adjust the simulation parameters and observe how particle motion changes in real time. Users can tune parameters and watch how the particle motion changes, including how attraction and collision behavior shift when forces are turned on and off. The second is a multiplayer competitive mode, in which multi-

[1] https://anonymous.4open.science/r/MD-Kivy-FBFB

ple participants compete on two opposing sides of a shared screen. This mode uses simplified hard-sphere physics to demonstrate the ideal gas law through gameplay.

MD-Kivy also supports optional hardware extensions. An Arduino accelerometer can be shaken by the user to simulate increasing particle speed and energy into the simulation. Padded floor controllers (using a Makey-Makey) allow for users to stomp and remove opponent's atoms in the competitive mode. Both of these peripherals aim to increase the physical and mental engagement of K-12 users beyond just touch interaction. These unique peripherals are optional; all features can still be operated with a touchscreen and keyboard.

4 Molecular Dynamics Model

MD-Kivy implements a two-dimensional particle simulation for exploring molecular motion through direct interaction. The model does not aim to reproduce a specific real material quantitatively. Users can adjust particle count, force strength, speed, and timestep, then watch how particle motion and grouping behavior change in response.

4.1 Single-Player Exploration Mode

The simulation offers two operating regimes. With intermolecular forces disabled, molecules move freely and reflect elastically from the display boundaries. Users can observe straight-line trajectories and wall collisions before introducing pairwise interactions. With forces enabled, each molecule pair interacts through a Lennard-Jones-inspired model.

The Lennard-Jones model [10] is commonly used to represent particles that repel strongly at very short distances and attract weakly at intermediate distances. Its standard potential energy form is

$$U(r) = 4\varepsilon\left[\left(\frac{\sigma}{r}\right)^{12} - \left(\frac{\sigma}{r}\right)^{6}\right]. \tag{1}$$

Here, r is the center-to-center distance between two molecules, ε controls the depth of the attractive well, and σ sets the effective molecular diameter. The repulsive r^{-12} term dominates at short range, while the attractive r^{-6} term dominates at intermediate range. The potential reaches its minimum at $r = 2^{1/6}\sigma$, which corresponds to the preferred separation between two interacting molecules.

Figure 2 illustrates the repulsive and attractive regimes of the Lennard-Jones interaction used in MD-Kivy.

The system prioritizes qualitative over quantitative fidelity; Lennard-Jones parameters scale to screen-space units, ensuring attraction and repulsion remain visually apparent during interaction. The implementation evaluates the analytic Lennard-Jones force, $F(r) = -dU/dr$, in reduced units, measuring intermolecular distance in molecular diameters: $r^* = r/\sigma_{\text{px}}$, where $\sigma_{\text{px}} = 2R\sigma$ converts the user-adjustable σ into pixels through the drawn molecular radius R. The pairwise force magnitude is

$$F^*(r^*) = \frac{24\varepsilon}{r^*}\left[2\left(\frac{1}{r^*}\right)^{12} - \left(\frac{1}{r^*}\right)^{6}\right]. \tag{2}$$

The force is applied in screen space by multiplying the radial force-over-distance term by the pixel displacement vector between the pair, which is equivalent to $F_{\text{screen}} = F^*\sigma_{\text{px}}$. The geometric factor σ_{px} (tens of pixels on a large-format display, giving an effective screen-space prefactor $24\,\sigma_{\text{px}} \approx 10^3$) is what produces visible motion at interactive timescales. Since this factor comes from measuring distances in molecular diameters, it scales with the display instead of being a hand-tuned constant; it changes only the

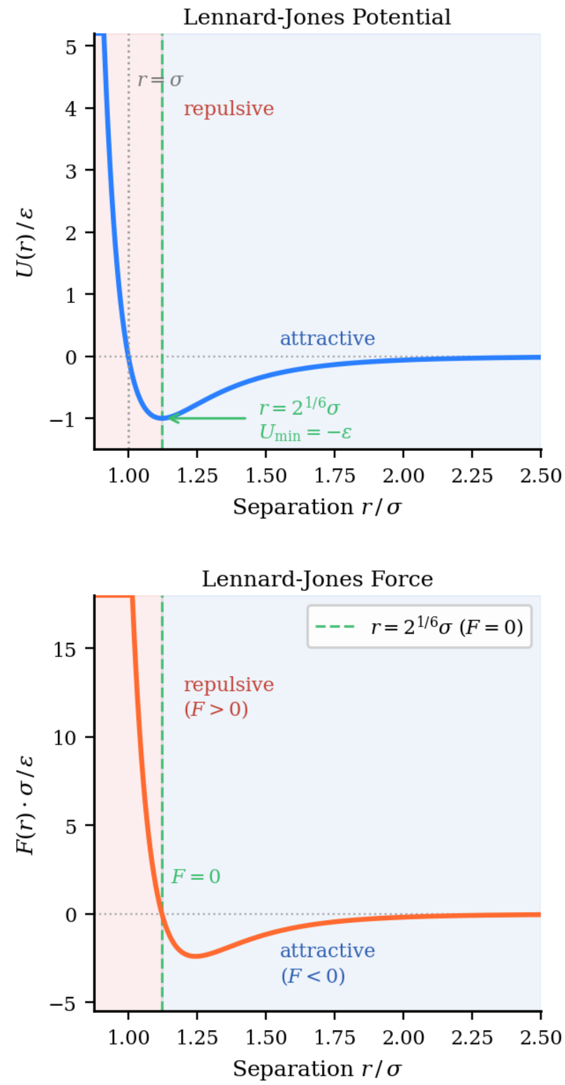

Figure 2: Normalized Lennard-Jones potential and radial force. The equilibrium separation occurs at $r = 2^{1/6}\sigma$.

visual timescale and preserves the qualitative Lennard-Jones structure, with the same equilibrium separation at $r^* = 2^{1/6}$ (equivalently $r = 2^{1/6}\sigma_{\text{px}}$).

Both ε and σ are adjustable at runtime (Figure 3). Increasing ε strengthens attraction and promotes clustering, while increasing σ extends the interaction range. The σ entering Eq. 2 through σ_{px} is the user-adjustable interaction parameter; collision resolution (Section 4.3) instead uses the sum of the molecular pixel radii, $r_i + r_j$, as the effective diameter, so overlapping molecules are always separated relative to their drawn size, regardless of the current σ setting.

Force contributions beyond 2.5σ are discarded; interactions at that range are negligible and the visible short-range behavior is unaffected.

Figure 4 illustrates why the cutoff and live computational load display matter as molecule count grows. The upper panel shows that the number of pairwise checks scales as $N(N-1)/2$: at $N = 20$ this is 190 pairs, at $N = 50$ it reaches 1,225, and at $N = 100$ it is 4,950. The lower log-log panel shows the theoretical pair count with and without a 2.5σ cutoff: beyond that range, most pairs contribute negligible force, so a neighbor-list implementation could

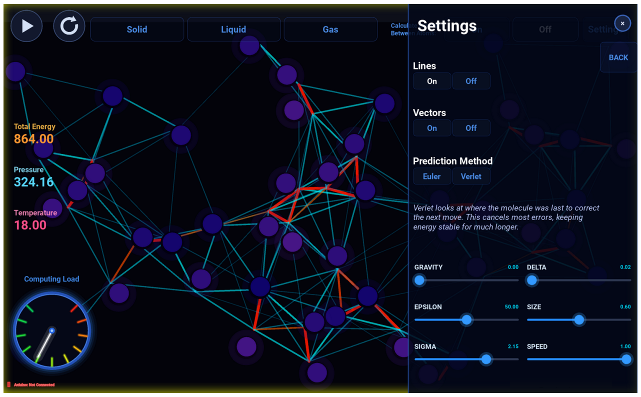

Figure 3: The single-player settings panel. Users switch between Euler and Velocity Verlet integration, adjust the Lennard-Jones parameters ε and σ, change the timestep, and toggle connection lines and velocity vectors. The thermodynamic measurements and the "Computing Load" indicator update in real time.

skip them and approach near-linear cost. MD-Kivy instead evaluates the full pairwise matrix and applies the cutoff as a mask (Section 8), so the cutoff curve represents the interaction locality of the model rather than the cost of the current implementation. The speedometer display communicates this scaling qualitatively in real time, giving users a visual signal that adding molecules increases computational cost faster than linearly.

4.2 Numerical Integration

MD-Kivy provides two numerical integration schemes for the single-player simulation. Since the system is implemented in screen-space units, the force term is treated as a screen-space acceleration after applying the scaling described above. The default integration scheme is a semi-implicit (symplectic) Euler update [2], which first applies the current net force to advance velocity and then uses the updated velocity to advance the position:

$$\mathbf{v}_{n+1} = \mathbf{v}_n + \mathbf{F}(\mathbf{x}_n)\Delta t, \tag{3}$$

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \mathbf{v}_{n+1}\Delta t. \tag{4}$$

This update is simple and fast, but it can accumulate visible energy error when the timestep is too large or when particles interact strongly at a short range. MD-Kivy also implements the Velocity Verlet integration [22] as a more stable alternative. This method first advances velocity by a half step,

$$\mathbf{v}_{n+\frac{1}{2}} = \mathbf{v}_n + \frac{1}{2}\mathbf{F}(\mathbf{x}_n)\Delta t, \tag{5}$$

then updates position,

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \mathbf{v}_{n+\frac{1}{2}}\Delta t, \tag{6}$$

and finally recomputes forces at the new positions before completing the velocity update:

$$\mathbf{v}_{n+1} = \mathbf{v}_{n+\frac{1}{2}} + \frac{1}{2}\mathbf{F}(\mathbf{x}_{n+1})\Delta t. \tag{7}$$

The timestep Δt is exposed as an interactive parameter. Users can observe that molecular dynamics has numerical stability limits, not just physical ones. With small timesteps, particle motion remains smooth, and energy behavior is more stable. With excessively large timesteps, the system can visibly diverge, producing rapid energy

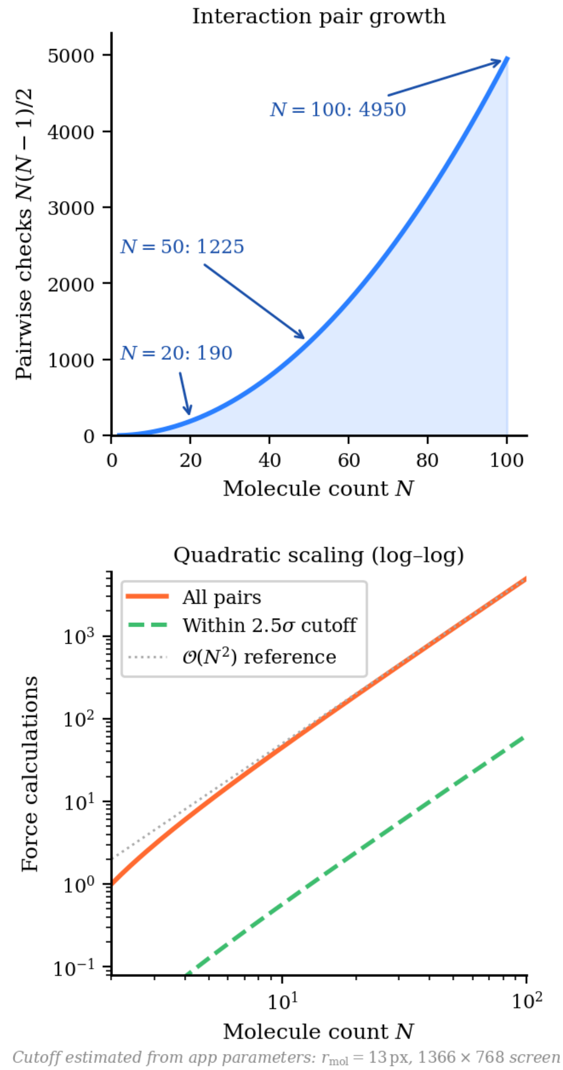

Figure 4: Pairwise interaction growth in the molecular dynamics simulation. The number of possible molecule pairs grows as $N(N-1)/2$, producing quadratic scaling as molecule count increases. The cutoff curve shows the pairs that remain physically relevant within 2.5σ; the current implementation still evaluates all pairs and applies the cutoff as a mask.

growth and unstable motion. To prevent unrecoverable blowup during close-range encounters, MD-Kivy clamps particle velocities after each update:

$$\mathbf{v} \leftarrow \min\left(1, \frac{v_{\max}}{|\mathbf{v}|}\right)\mathbf{v}. \tag{8}$$

The clamp preserves direction and caps speed magnitude, and the display stays responsive during close-range encounters. This sensitivity to timestep is also why Velocity Verlet is offered as an alternative to symplectic Euler; it maintains more stable energy behavior at larger timesteps, giving users a way to compare integrator behavior directly.

4.3 Collision Resolution

Overlap between particles is resolved differently depending on whether intermolecular forces are active. When forces are disabled, overlapping pairs are resolved with a standard two-dimensional elastic hard-sphere model. The collision update modifies the velocity components along the contact normal according to conservation of momentum and kinetic energy, while the tangential components remain unchanged. A small positional correction displaces each molecule by 51% of the overlap distance. This value is just high enough to leave a guaranteed gap, preventing floating-point rounding from retriggering the collision check. It remains small enough to avoid visible jitter. The resulting separation ensures the collision check does not trigger repeatedly on the subsequent frame.

When Lennard-Jones forces are active, the elastic exchange is replaced by a softer resolution. Overlapping pairs are separated to a fixed screen-space distance of $2^{1/6}(r_i + r_j)$, where r_i and r_j are the molecular radii. At the default setting $\sigma = 1$ this coincides with the Lennard-Jones minimum, where the pair potential energy is $-\varepsilon$; for other values of σ, it acts as a radius-based stabilization rule near, but not exactly at, the minimum. Simultaneously, the relative normal velocity between the pair is zeroed by setting both molecules to their shared center-of-mass normal velocity, so the pair does not immediately separate. Tangential velocity components are left unchanged. Clusters remain visually stable, and particles do not lock together at unrealistically small separations.

4.4 Multiplayer Competitive Mode

The competitive mode uses a deliberately simplified physical model to prioritize responsiveness and competitive gameplay. Lennard-Jones forces are disabled because pairwise force calculations at high molecule counts would introduce latency during competitive play (Table 1). Molecules behave as hard spheres undergoing elastic collisions with each other and with the arena walls. A central divider separates the two competing sides. Multiple participants can play on each side. Group members tap their half of the screen in quick succession to build their side's count. A single Makey Makey pad can also be shared by several players joined in a conductive chain, letting a whole group control one side together. Participants tap anywhere on their half of the screen to place a new molecule at the touch position with a randomized velocity; each spawned molecule adds kinetic energy, raising that side's temperature-like reading. Each Makey Makey controller acts as a sabotage tool: pressing the pad removes one molecule from the opponent's side. The removal triggers a bubble-pop animation, an expanding ring grows from the molecule's position outward, and eight small droplets fly in evenly spaced directions, all rendered in the opposing team's color.

The first side to heat its half to the target reading of 373.15, displayed in kelvin-like units as a nod to the boiling point of water, wins. At that point, the divider is removed and the molecules expand freely in the combined arena (Figure 5). Before the molecules diffuse throughout the combined area, a visual shattering animation marks the removal of the divider. In addition to this, the simplified model emphasizes gas-like motion and free expansion, which users experience as a direct result of winning the competition. The clear win condition and direct control over spawning give players defined goals and a sense of control, conditions linked to focused, engaged play [9].

4.5 Visual Encoding

Across both modes, molecules are colored by instantaneous speed. In single-player mode, slow particles appear as dark blue-purple, and faster particles shift toward bright pink-magenta. In Competitive mode, speed modulates the brightness and saturation of each team's color; team identity remains intact while relative motion remains visible. Velocity arrows are available as an overlay in single-player mode, showing direction and relative speed during collisions and force interactions.

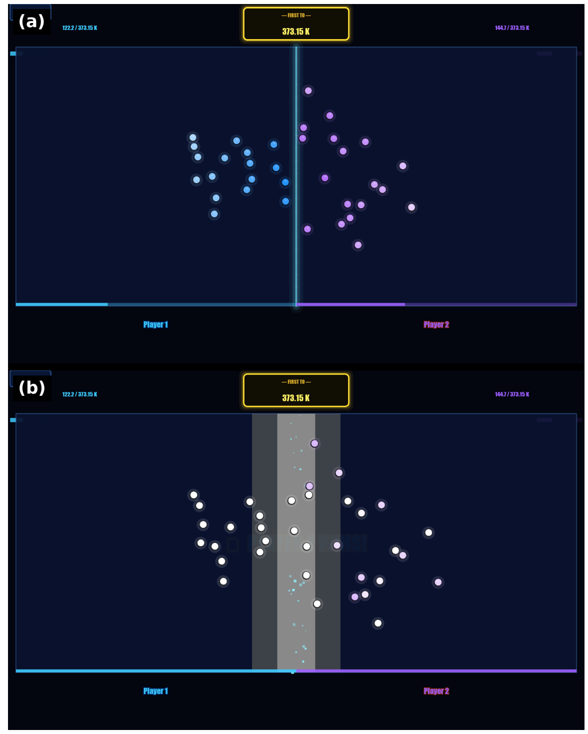

Figure 5: Multiplayer competitive mode. (a) Two teams, blue and purple, race to heat their half of the divided arena by tapping to spawn molecules. (b) When one side reaches the target reading of 373.15, the divider shatters, and the molecules undergo free expansion into the newly combined arena.

5 Input Modalities

MD-Kivy supports four input methods: touchscreen, keyboard, Arduino accelerometer input, and Makey Makey controllers. These methods can coexist; the application detects available hardware at startup and enables the corresponding inputs. Offering several familiar input methods reflects common interaction-design principles. When controls map naturally to on-screen effects and the system provides immediate feedback, people can use it without instruction [15]. Familiar controls and a choice of input methods also reduce the learning curve for short, self-directed use [9, 20]. In MD-Kivy, shaking the accelerometer to add energy or touching a conductive pad to spawn a molecule links each action to a visible change in the simulation.

Touchscreen. The primary interface is a large-format capacitive touchscreen. Users add molecules by tapping directly on the simulation area, adjust parameters using sliders and buttons on the screen, and move between modes through Kivy touch events. Users change the simulation directly and see the effect on particle motion immediately.

Keyboard. A keyboard interface is included as a fallback input method and for demonstration use. In competitive mode, keyboard input supports player name entry and controller-style actions. The name-entry lobby binds directly to Kivy's `Window.on_key_down` event instead of relying on a standard `TextInput` widget, avoiding focus issues that can occur on Linux touchscreen setups.

Arduino accelerometer. MD-Kivy can also use an Arduino-based accelerometer that streams three-axis motion data through USB serial or a wireless TCP bridge (Figure 6). The reader accepts multiple sensor formats and runs without blocking the simulation loop. Shaking the device injects kinetic energy into the particle system, while tilt can be mapped to a gravity-like parameter. Physical

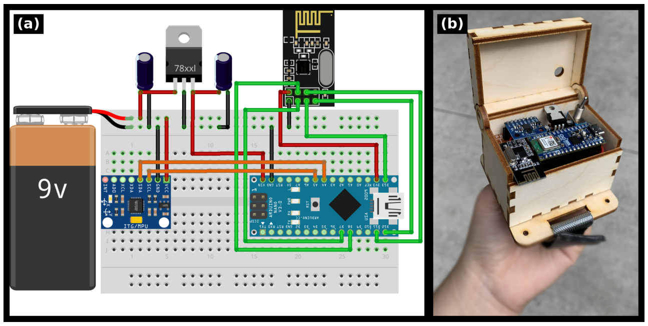

Figure 6: Arduino accelerometer controller. (a) Breadboard wiring of the wireless transmitter: a 9 V supply and voltage regulator power an Arduino Nano that reads an inertial sensor, and streams motion data over an nRF24L01 radio. (b) The assembled unit in a laser-cut wooden enclosure with a power switch.

motion maps directly to visible changes in the particle system.

Makey Makey controllers. Makey Makey is a platform for turning everyday conductive objects into tangible controllers [3]. Its boards appear to the operating system as USB keyboards. MD-Kivy detects compatible boards and displays an on-screen key-mapping legend when a controller is available. This lets users trigger simulation actions through physical pads, in addition to the touchscreen controls.

Supporting two identical Makey Makey boards in two-player mode requires additional input handling. Although Makey Makey boards are physical controllers, the operating system treats them as USB keyboard devices. When two boards are connected, both send the same key codes. Standard Kivy keyboard events carry only the key code, so there is no way to tell which board produced a given press.

MD-Kivy resolves this by reading input from each device separately using `evdev`. Instead of relying on the merged keyboard event stream, the application scans the Linux input devices in `/dev/input/` and selects the keyboard interface associated with each Makey Makey board. The first detected board is assigned to the left player, and the second is assigned to the right player. The same key code now triggers different actions depending on which physical device produced it.

Each controller is monitored by a separate background thread. When a key-down event is detected, the event is debounced so that repeated hardware signals from a single touch do not spawn multiple molecules. The application also ignores input during a brief startup period to prevent stale device states from triggering actions before the simulation begins. Accepted events are passed back to the Kivy main thread via `Clock.schedule_once`, keeping simulation updates single-threaded.

6 System Implementation

MD-Kivy is implemented in Python using the Kivy framework and runs on Linux. The application is organized as a multi-screen interface using Kivy's `ScreenManager`. The main screens include a landing screen, an instructional start screen, a single-player simulation screen, a competitive name-entry lobby, a competitive arena, and a leaderboard. Screen transitions use `NoTransition` to prevent touch events from carrying over during mode changes.

Simulation loop. The physics update runs at a base rate of 30 Hz using Kivy's `Clock.schedule_interval`. A user-adjustable speed multiplier modifies the update interval so users can slow down or speed up the simulation during interaction. To reduce overhead when many molecules are present, thermodynamic measurement and velocity-arrow overlays are refreshed every five frames. A lightweight background governor checks runtime load every 0.5 seconds and can increase the update interval when the frame budget is exceeded. Explicit garbage collection is triggered every 30 seconds to limit memory growth from discarded canvas instructions.

Rendering. Rendering is performed directly with Kivy OpenGL canvas instructions without an external graphics engine. Molecules, velocity arrows, boundary walls, and visual overlays are represented using Kivy `Ellipse`, `Line`, and `Color` instructions. As particle positions and visual overlays change, the corresponding Kivy canvas instructions are updated to match the current simulation state.

Computational load feedback. MD-Kivy includes a speedometer display that provides a qualitative estimate of simulation load. The interface labels this indicator "Computing Load." The score combines molecule count scaled empirically as $N^{1.2}$, the epsilon parameter, simulation speed, and total kinetic energy. When Lennard-Jones forces are active, the combined score is multiplied by 2 to account for the added force calculation performed for each molecule pair. The metric operates as a perceptual heuristic, as opposed to a formal execution profiler. It is tuned to ensure the display responds dynamically across the practical range of molecule counts, bypassing a direct tracking of the $\mathcal{O}(N^2)$ pair growth shown in Figure 4. Arduino accelerometer activity contributes an additional term when the physical controller is connected. The resulting value is smoothed with a fixed rise and decay rate before driving the speedometer needle. The needle sweeps a colored arc from green (low load) through yellow to red (high load), similar to a car tachometer. As users add molecules and the needle climbs toward red, the display shows why large-scale molecular dynamics simulations need high-performance computers instead of a single desktop.

Competitive-mode data persistence. Competitive mode results are stored locally in a JSON file. Each entry records the winner's name, the loser's name, the winning side, the date, and the time of the match. The leaderboard retains the most recent 100 matches and is updated at the end of each competitive session, so competitive history persists across repeated uses.

7 Deployment and Design Observations

MD-Kivy was developed and iteratively refined within its intended deployment environment, the Visualization Laboratory at the Texas Advanced Computing Center (TACC). The laboratory regularly hosts K–12 student groups, university classes, educators, and public visitors. During development, the system was installed on the facility display and periodically demonstrated during tours as an example of molecular dynamics and computational science. This setting shaped several design requirements. The simulation needed to be visually readable from across the room, understandable during a short tour stop, and engaging for visitors of different ages and scientific backgrounds without lengthy setup or instruction.

Throughout development, successive prototypes were demonstrated during guided tours and educator visits within the TACC Visualization Laboratory. Feedback from tour audiences, educators, and outreach staff informed successive design iterations. This process led to numerous improvements, including larger text and controls for increased visibility, reorganized interface elements, clearer terminology and color mappings, the addition of a computational workload visualization, and a simplified display that reduced visual clutter while preserving key educational content.

After these interface revisions, two high school chemistry teachers served as consultants to review the completed system in person. Their feedback suggested that the interface was well suited for educator-led demonstrations and public outreach. They also identified opportunities to extend the exhibit beyond informal learning by developing classroom activities aligned with high school chemistry curricula. These curriculum materials are currently under development in collaboration with practicing teachers.

The observations presented here constitute a formative evaluation of the exhibit design rather than a controlled assessment of learning. They suggest that MD-Kivy is feasible for deployment in informal learning environments and for educator-led demonstrations while leaving questions of educational effectiveness to future work. Planned studies will examine how visitors interact with the visualization, what conceptual understanding they develop, and how the exhibit can be integrated into high school chemistry instruction.

8 Discussion and Limitations

MD-Kivy was designed as an educational visualization rather than a research-grade molecular dynamics simulator. Designing an interactive educational visualization requires balancing scientific fidelity with usability, responsiveness, and conceptual clarity. MD-Kivy intentionally simplifies many aspects of molecular dynamics in order to support real-time interaction, collaborative exploration, and public engagement. The following discussion outlines these design tradeoffs and the resulting limitations of the current system.

Simplified Physical Model. The 2D simulation models identical particles with uniform radii. Typical runs contain tens to a few hundred particles, far from the thermodynamic limit. The boundaries are reflective walls, not the periodic boundary conditions common in research MD, so wall effects are large at these particle counts. All quantities are computed in screen-space units. The displayed energy and temperature are derived from mean scaled kinetic energy, while the displayed pressure reflects momentum magnitude. Therefore, none of these displayed values corresponds to physical units. The gravity control is also included strictly as a teaching tool, given that gravitational forces are negligible at the molecular scale.

We accept these simplifications because the model's primary goal is to illustrate basic qualitative trends: attraction triggers clustering, energy inputs increase temperature, and larger particle counts require more computational power. The principal risk is that users might mistakenly apply macro-scale, on-screen intuitions to actual physical scales. Consequently, both exhibit signage and active facilitation must clearly communicate the qualitative nature of the model.

Interactive Stability. Several of the stabilizing mechanisms break energy conservation. The velocity clamp (Eq. 8) removes kinetic energy during close encounters. When active, the soft collision resolution rules place particle pairs at a fixed separation near the potential minimum and reset their relative normal velocity to zero, a process that also dissipates energy. The solid preset applies a mild velocity-rescaling thermostat. Users can also add energy at any time through the interface controls or an Arduino accelerometer.

Consequently, the system fails to sample a well-defined statistical ensemble, meaning the energy measurements capture overall behavioral trends instead of a perfectly conserved quantity. The stability indicator communicates this state to users, warning them when factors like the integrator choice, timestep, energy drift, or particle counts threaten accuracy.

Real-Time Performance. Force evaluation uses NumPy to build the full pairwise distance matrix and applies the 2.5σ cutoff as a mask, so computational cost and memory still scale as $\mathcal{O}(N^2)$ regardless of how many pairs actively interact. Table 1 quantifies this scaling: on the development system, the physics step stays within the 33 ms budget of the 30 Hz update loop up to roughly 300 molecules with forces enabled, and enabling Lennard-Jones forces adds 40-50% to the step time at high molecule counts. Under sustained load, the background governor extends the update interval, which keeps the interface responsive but makes simulation speed hardware-dependent.

Table 1: Mean physics-step wall time and implied maximum update rate versus molecule count N, with Lennard-Jones forces disabled and enabled (semi-implicit Euler, $\varepsilon = 50$, $\sigma = 1$, 1366×768 arena; 300 steps after 60 warm-up steps; rendering excluded). Measured on the development system (Intel Core i7-1250U, 16 GB RAM, Kivy 2.3.1, NumPy 2.2.3).

	Forces off		Forces on	
N	ms/step	max. Hz	ms/step	max. Hz
50	2.4	416	2.9	341
100	5.1	198	6.4	157
200	11.9	84	17.5	57
300	23.0	43	32.8	30
400	44.0	23	62.3	16

Gameplay and Embodied Interaction. Competitive mode disables Lennard-Jones forces and uses hard-sphere collisions to keep latency low at high molecule counts. Its molecule-removal mechanic deletes molecules instantly, which has no physical analogue. Gameplay, therefore, shows the dynamics of a confined gas, not intermolecular chemistry; the physically meaningful event is the free expansion when the divider drops.

Deployment Constraints and Accessibility. The exhibit targets a Linux touchscreen, and two-player controller support depends on that choice: telling two identical Makey Makey boards apart requires reading raw events from `/dev/input/`, which needs input-device permissions and is not available on other operating systems, where the boards merge into a single keyboard stream. Arduino motion data arrives at about ten samples per second and is smoothed before it reaches the simulation, so shaking feedback has noticeable latency. The speed-to-color encoding varies luminance monotonically as well as hue; under simulated protanopia, deuteranopia, and tritanopia [12], the endpoints of the ramp retain a WCAG-style luminance contrast ratio [24] above 6:1, so relative speed remains readable, although a formal accessibility evaluation with users has not been conducted. The current build has no audio channel, which limits accessibility for visitors with low vision.

9 Conclusion and Future Work

MD-Kivy demonstrates how interactive educational visualization can make the fundamental ideas underlying molecular dynamics and scientific computing accessible to public audiences. The system does not reproduce research-grade molecular dynamics; it emphasizes conceptual understanding through direct manipulation, letting visitors explore how intermolecular forces, energy, and computational complexity influence system behavior. By combining real-time simulation, tangible interaction, and collaborative exploration within a public exhibit, MD-Kivy illustrates how visualization can support both informal STEM learning and public engagement with computational science.

The formative observations and teacher consultations described in Section 7 suggest that the system is suitable for educator-led demonstrations and public deployment, and they informed several iterations of the interface design. However, these observations were not intended to evaluate educational effectiveness, and we have not yet conducted controlled studies measuring visitor engagement, conceptual understanding, learning outcomes, or the persistence of misconceptions.

Future work will include structured evaluations of visitor learning and engagement, a formal accessibility study, and continued collaboration with high school chemistry teachers to develop curriculum materials that extend the exhibit into classroom instruction. Additional technical enhancements include support for multiple molecular species, periodic boundary conditions, and the optional display of approximate physical units alongside the existing screen-space values.

ACKNOWLEDGMENTS

Withheld for double-blind review.

REFERENCES

[1] W. K. Adams et al. A study of educational simulations part i: Engagement and learning. *Journal of Interactive Learning Research*, 19(3):397–419, 2008. 2

[2] M. P. Allen and D. J. Tildesley. *Computer Simulation of Liquids*. Oxford University Press, 2nd ed., 2017. 4

[3] Beginner's Mind Collective and D. Shaw. Makey makey: Improvising tangible and nature-based user interfaces. In *Proceedings of the Sixth International Conference on Tangible, Embedded and Embodied Interaction (TEI '12)*, pp. 367–370. ACM, 2012. doi: 10.1145/2148131.2148219 6

[4] Center for Polymer Studies, Boston University. The virtual molecular dynamics laboratory. http://polymer.bu.edu/vmdl/, 2004. 2

[5] W. Christian and F. Esquembre. Modeling physics with easy java simulations. *The Physics Teacher*, 45(8):475–480, 2007. doi: 10.1119/1.2798358 2

[6] J. N. da Silva Júnior, M. d. S. C. Teotônio, R. C. S. Jucá, G. de Lima Castro, and A. J. Melo Leite Júnior. 1925–2024: One century of educational games in chemistry. *Journal of Chemical Education*, 102(4):1492–1510, 2025. doi: 10.1021/acs.jchemed.4c01238 2

[7] S. Elmali, L. Freund, and R. Tiemann. A systematic literature review of game-based approaches in chemistry education (2014–2024). *Chemistry Education Research and Practice*, 27:31, 2026. doi: 10.1039/d5rp00248f 2

[8] T. D. Goddard et al. Ucsf chimerax: Meeting modern challenges in visualization and analysis. *Protein Science*, 27(1):14–25, 2018. doi: 10.1002/pro.3235 2

[9] C. Hodent. *The Gamer's Brain: How Neuroscience and UX Can Impact Video Game Design*. CRC Press, 2018. 5

[10] J. E. Lennard-Jones. On the determination of molecular fields. ii. from the equation of state of a gas. *Proceedings of the Royal Society of London. Series A*, 106(738):463–477, 1924. doi: 10.1098/rspa.1924.0082 3

[11] M. Ma and A. Oikonomou, eds. *Serious Games and Edutainment Applications, Volume II*. Springer, 2017. doi: 10.1007/978-3-319-51645-5 2

[12] G. M. Machado, M. M. Oliveira, and L. A. F. Fernandes. A physiologically-based model for simulation of color vision deficiency. *IEEE Transactions on Visualization and Computer Graphics*, 15(6):1291–1298, 2009. doi: 10.1109/TVCG.2009.113 7

[13] X. Martinez and M. Baaden. Unitymol prototype for FAIR sharing of molecular-visualization experiences. *Acta Crystallographica Section D: Structural Biology*, 77(6):746–754, 2021. doi: 10.1107/S2059798321002941 2

[14] National Research Council. *Learning Science in Informal Environments: People, Places, and Pursuits*. The National Academies Press, Washington, DC, 2009. doi: 10.17226/12190 2

[15] D. A. Norman. *The Design of Everyday Things*. Basic Books, revised and expanded ed., 2013. 5

[16] M. O'Connor et al. Sampling molecular conformations and dynamics in a multiuser virtual reality framework. *Science Advances*, 4(6):eaat2731, 2018. doi: 10.1126/sciadv.aat2731 2

[17] J. Schnitker. Molecular-level simulations as a chemistry teaching tool. In M. D. Ellison and T. A. Schoolcraft, eds., *Advances in Teaching Physical Chemistry*, number 973 in ACS Symposium Series, chap. 12. American Chemical Society, Washington, DC, 2008. doi: 10.1021/bk-2008-0973.ch012 2

[18] D. V. Schroeder. Interactive molecular dynamics. *American Journal of Physics*, 83(3):210–218, 2015. doi: 10.1119/1.4901185 2

[19] S. Seritan, Y. Wang, J. E. Ford, A. Valentini, T. Gold, and T. J. Martínez. Interachem: Virtual reality visualizer for reactive interactive molecular dynamics. *Journal of Chemical Education*, 98(11):3486–3492, 2021. doi: 10.1021/acs.jchemed.1c00654 2

[20] S. Stahlke and P. Mirza-Babaei. Control freaks. In *The Game Designer's Playbook: An Introduction to Game Interaction Design*, chap. 5. Oxford University Press, 2022. doi: 10.1093/oso/9780198845911.003.0005 5

[21] J. E. Stone, J. Gullingsrud, and K. Schulten. A system for interactive molecular dynamics simulation. In *Proc. 2001 Symposium on Interactive 3D Graphics (I3D '01)*, pp. 191–194, 2001. doi: 10.1145/364338.364398 2

[22] W. C. Swope, H. C. Andersen, P. H. Berens, and K. R. Wilson. A computer simulation method for the calculation of equilibrium constants for the formation of physical clusters of molecules: Application to small water clusters. *The Journal of Chemical Physics*, 76(1):637–649, 1982. doi: 10.1063/1.442716 4

[23] J. M. Wing. Computational thinking. *Communications of the ACM*, 49(3):33–35, 2006. doi: 10.1145/1118178.1118215 2

[24] World Wide Web Consortium (W3C). Web content accessibility guidelines (wcag) 2.1. W3C Recommendation, https://www.w3.org/TR/WCAG21/, 2018. 7